Universidad Peruana Cayetano Heredia
Facultad de Ciencias e Ingeniería
Departamento de Ciencias Exactas

Programación Avanzada 2025

Lab. 11.1. Algoritmos de Búsqueda

Junio 04, 2025

Cree una carpeta (folder) en el disco D, nómbrela con su apellido paterno seguido de su código. Ejemplo: LOPEZ12345

Búsqueda es el proceso algorítmico de encontrar un ítem particular en una colección de ítems. Un ítem puede ser una palabra clave en un archivo, un registro en una base de datos, un nodo en un árbol o un valor en una lista, etc.

Búsqueda y ordenación tienen una amplia aplicación, es así que, a lo largo del tiempo, se ha dedicado mucho esfuerzo para descubrir los algoritmos más rápidos de búsqueda y ordenación. Por lo que, el análisis de la performance de estos algoritmos es crítico.

Búsqueda de un mínimo

La función **min** de Python devuelve el valor mínimo de una lista. A fin de estudiar la complejidad del algoritmo, desarrollemos una versión alternativa que devuelve la *posición* del valor mínimo. El algoritmo asume que la lista no está vacía y los ítems están en un orden arbitrario. El algoritmo inicia asumiendo que la primera posición es la del mínimo. He aquí el código de la función `nuestroMin` que implementa el algoritmo:

```
def nuestroMin(lyst):
      """Devuelve la posición del valor mínimo."""
      minpos = 0
      current = 1
      while current < len(lyst):
            if lyst[current] < lyst[minpos]:
                  minpos = current
            current += 1
      return minpos
```

1. ¿Cuál es la complejidad del algoritmo presentado?, explique su respuesta.

Búsqueda lineal en una lista

Cuando los elementos de una lista no están ordenados y queremos buscar un valor particular, necesitamos recorrer la lista hasta encontrar el valor deseado, en el peor de los casos podría significar recorrer toda la lista completa. Este tipo de algoritmo es conocido como **búsqueda lineal** en una secuencia desordenada. He aquí el código de la función de búsqueda lineal:

```
def linearSearch(target, lyst):
      """Devuelve la posición de target si lo encuentra o -1 en cualquier otro
      caso."""
      position = 0
      while position < len(lyst):
            if target == lyst[position]:
                  return position
            position += 1
      return -1
```

Desempeño en el mejor de los casos, en el peor de los casos, y el caso intermedio.

El desempeño de algunos algoritmos depende de la ubicación de los datos. El algoritmo de búsqueda lineal realiza un menor esfuerzo si el objetivo se encuentra al comienzo de la lista que al final de la lista. Para estos tipos de algoritmos es recomendable determinar la performance en el mejor de los casos, en el peor de los casos, y en el caso intermedio.

En general, un análisis exhaustivo de la complejidad de un algoritmo divide su comportamiento en esos tres tipos de casos:

a) **Mejor de los casos**: ¿En qué circunstancias el algoritmo realiza la menor cantidad de trabajo? ¿cuál es la complejidad del algoritmo en este caso?
b) **Peor de los casos**: ¿En qué circunstancias el algoritmo realiza la mayor cantidad de trabajo? ¿cuál es la complejidad del algoritmo en este caso?
c) **Caso intermedio**: ¿En qué circunstancias el algoritmo realiza la cantidad típica de trabajo? ¿cuál es la complejidad del algoritmo en este caso?

En general, nos preocupamos más de la performance en el caso intermedio y en el peor de los casos.

2. Determinar la complejidad del algoritmo de búsqueda lineal:
   a) En el peor de los casos
   b) En el mejor de los casos
   c) En el caso intermedio

Búsqueda binaria en una lista

La búsqueda binaria se aplica si la lista está previamente ordenada. Asumamos que la lista está ordenada en forma ascendente, el algoritmo de búsqueda va directo a la posición del elemento que se encuentra en la mitad y lo compara con el valor objetivo, si son iguales, devuelve la posición. De lo contrario, si el objetivo es menor que el ítem actual, el algoritmo acorta el espacio de búsqueda - a la porción de la lista antes de la posición del medio. Si el objetivo es mayor que el ítem actual, el algoritmo acorta el espacio de búsqueda – a la porción de la lista después de la posición del medio. El proceso de búsqueda termina cuando el objetivo es encontrado o cuando la posición de inicio actual es mayor que la posición de término actual.

He aquí el código para la función de búsqueda binaria.

```
def binarySearch(target, lyst):
      """Returns the position of the target item if found,
      or -1 otherwise."""
      left = 0
      right = len(lyst) - 1
      while left <= right:
            midpoint = (left + right) // 2
            if target == lyst[midpoint]:
                  return midpoint
            elif target < lyst[midpoint]:
                  right = midpoint - 1 # Search to left
            else:
                  left = midpoint + 1 # Search to right
      return -1
```

3. Determinar la complejidad del algoritmo de búsqueda binaria:
   a) En el peor de los casos
   b) En el mejor de los casos
   c) En el caso intermedio

4. Suponga que una lista contiene los siguientes valores:
   20 44 48 55 62 66 74 88 93 99
   Realice la traza de las variables **left**, **right**, y **midpoint** si se aplica el algoritmo de búsqueda binaria a la lista dada.
   a) Analice para el valor objetivo 90
   b) Analice para el valor objetivo 44

5. Elabore un programa de prueba para comparar las distintas técnicas de búsqueda en una lista de números. Mida el tiempo que demora cada algoritmo para encontrar un objetivo.
   a) Los algoritmos a comparar son: búsqueda lineal usando `while`, búsqueda lineal usando `for`, búsqueda binaria, y la función `index()` de Python.
   b) Asuma que la lista sea, **`L = list(range(10000001))`**
   c) Verifique para un valor que se encuentre cerca al comienzo de la lista
   d) Verifique para un valor que se encuentre cerca al medio de la lista
   e) Verifique para un valor que se encuentre cerca al final de la lista

Opcional:

6. Implemente el algoritmo de búsqueda binaria usando recursión.

7. Dada una lista de números, encontrar el elemento que aparece la mayor cantidad de veces.
   a) Aplicando fuerza bruta. Determine la complejidad para este caso.
   b) Ordenando la lista previamente. Determine la complejidad para este caso.

Guarde todos vuestros programas en una carpeta con el nombre su **Apellido** paterno seguido de vuestro **DNI**, luego comprima esta carpeta. Envíe este archivo a Katherine Navarro katherine.navarro@upch.pe especificando como asunto **Lab11.1**.